UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET

Poređenje algoritma za preporuku (Two Tower, Matrix Factorization, Deep Learning)

Student:

Natalija Pavlović 2125

Mentor:

doc. dr Miloš Bogdanović

Niš 2026. godina

Sadržaj

1. Uvod 3
2. Sistemi za preporuku 4
3. Analiza projekta 7
   3.1. Analiza skupa podataka 7
4. Matrix factorization 12
   4.1. Praktična implementacija Matrix factorization modela 13
   4.2. Rezultati treniranja modela 14
5. Two Tower 15
   5.1. Praktična implementacija Two Tower modela 15
   5.2. Rezultati treniranja modela 16
6. Deep learning 18
   6.1. Praktična implementacija Deep Learning modela 18
   6.2. Rezultati treniranja modela 19
7. Poređenje 20
   7.1. Poređenje preporuka anime serija 21
   7.2. Brzina treniranja modela 22
8. Zaključak 23
9. Literatura 24

1. Uvod

Savremene digitalne platforme generišu ogromne količine podataka o korisnicima i njihovim interakcijama sa sadržajem. Samo u toku jednog dana milijarde korisnika ostavljaju digitalne tragove kroz pretrage, klikove, kupovine i ocene, stvarajući složenu mrežu informacija koja, bez odgovarajućih alata ostaje neiskorišćena. U takvom okruženju postaje sve teže pronaći sadržaj koji je relevantan, koristan i prilagođen individualnim interesovanjima. Upravo iz te potrebe razvijeni su sistemi za preporuku - algoritamski modeli čiji je osnovni cilj personalizacija digitalnog iskustva.

Sistemi za preporuku danas predstavljaju ključnu komponentu savremenih platformi za elektronsku trgovinu, streaming servise, društvene mreže i obrazovne sisteme. Njihova uloga nije samo u povećanju korisničkog zadovoljstva već i u optimizaciji poslovnih rezultata kroz povećanje angažovanja i prodaje.

Oni koriste različite izvore podataka kako bi pronašli sadržaje od interesa za konkretnog korisnika. Entiteti kojima se daje preporuka nazivaju se korisnicima (*eng. users*), dok je predmet preporuke označen kao objekat/stavka (*eng. item*). Objekti mogu biti proizvodi, filmovi, pesme itd., sve što platforma nudi. U osnovi implementacije sistema za preporuku stoji pretpostavka da su ranija interesovanja i sklonosti korisnika dobar pokazatelj budućeg ponašanja, ali i da korisnici sa sličnim ukusima pokazuju slične obrasce ponašanja. Upravo ova dva načela, individualna istorija i kolektivna sličnost, čine temelj na kome su izgrađeni gotovo svi savremeni pristupi ovoj oblasti.

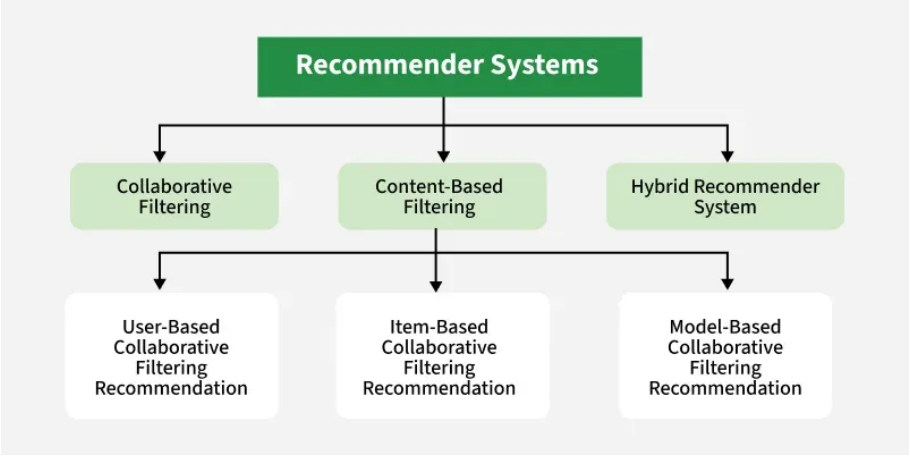
Ovaj rad ima za cilj poređenje tri pristupa izgradnji sistema za preporuku: Matrix Factorization, Two Tower modela i savremenih Deep Learning modela. Analiza će obuhvatiti teorijske osnove, način rada, prednosti i ograničenja svakog pristupa, kao i njihovu praktičnu primenu u realnim sistemima.

2. Sistemi za preporuku

Sistemi za preporuku pokušavaju da odgovore na jedno naizgled jednostavno pitanje: „Šta bi korisniku moglo da se dopadne?" Iza tog pitanja krije se čitav niz algoritama koji analiziraju korisničko ponašanje i na osnovu njega pokušavaju da predvide buduća interesovanja. Nije dovoljno samo znati šta je korisnik kupio ili pogledao. Važno je razumeti obrasce, kontekst i veze između različitih interakcija.

Postoje tri osnovna pristupa u izgradnji ovakvih sistema:

1. Kolaborativni sistemi polaze od ideje da ljudi sličnih ukusa biraju slične stvari. Ako su dva korisnika do sada voleli iste filmove, verovatno će se složiti i oko sledećeg. Ovaj pristup ne zahteva nikakvo znanje o samom sadržaju koji se preporučuje već se oslanja isključivo na istoriju interakcija korisnika.
2. Sistemi zasnovani na sadržaju umesto da gledaju šta drugi korisnici vole, oni analiziraju same stavke i preporučuju one koje su slične stvarima koje je korisnik ranije koristio ili ocenio pozitivno. Ako korisnik voli akcione filmove sa određenim rediteljem sistem će preporučiti druge filmove koji dele te karakteristike.
3. Hibridni sistemi kombinuju oba pristupa kako bi iskoristili prednosti svakog od njih i prevazišli njihove slabosti i upravo oni su danas najzastupljeniji u praksi. Gotovo svaka ozbiljna platforma koristi neki oblik hibridnog pristupa.



Slika 1. Vrste sistema za preporuku

Izgradnja dobrog sistema za preporuku nije jednostavna i u praksi se javljaju izazovi koji nisu odmah očigledni. Jedan od najčešćih je problem retkosti podataka. Ukoliko imamo platformu sa milionima korisnika i hiljadama proizvoda postoji šansa da većina korisnika će interagovati sa samo malim delom tog kataloga. Ogromna većina mogućih kombinacija korisnik-proizvod ostaje

potpuno neistražena. Sistemu jednostavno nedostaje dovoljno informacija da bi pouzdano procenio šta bi određenom korisniku moglo da se dopadne.

Drugi čest izazov je takozvani „hladni start". Hladni start se dešava kada novi korisnik tek napravi nalog i sistem ne zna ništa o njegovim preferencijama i teško mu je ponuditi relevantne preporuke. Isti problem se javlja i kada platforma doda novu stavku koja još nema dovoljno ocena ili interakcija.

Ova dva scenarija ,novi korisnik i nova stavka, predstavljaju klasičan problem sa kojim se suočavaju gotovo svi sistemi za preporuku.

Postoje dve vrste podataka koji sistemi za preporuku koriste:

1. Eksplicitni podaci su oni koje korisnik svesno ostavlja: ocena filma sa zvezdicama, like/dislike na objavi, komentar ili direktno dodavanje proizvoda na listu želja. Ovakvi podaci su izuzetno vredni jer jasno prikazuju korisnikove preference, ali problem može biti da ih većina korisnika retko ostavlja. Ljudi najčešće ne ocenjuju filmove koje pogledaju, ne pišu recenzije za svaki proizvod koji kupe i ne lajkuju svaki sadržaj koji im se svidi. Eksplicitnih podataka u praksi nikada nije dovoljno.
2. Implicitni podaci su suptilniji: koliko dugo si gledao neki video, koje si proizvode pregledao ali nisi kupio, šta si preskočio. Implicitni podaci su daleko bogatiji i dostupniji, ali ih je teže interpretirati. Ako je korisnik nešto pogledao ne znači nužno da mu se svidelo. Upravo zato moderni sistemi pokušavaju da kombinuju obe vrste podataka kako bi dobili potpuniju sliku o korisniku.

Sistem za preporuku pokušava da modeluje funkciju koja za svakog korisnika u i svaku stavku i predviđa ocenu relevantnosti:

$$f(u, i) = \widehat{r_{ui}}$$

gde $\widehat{r_{ui}}$ predstavlja predviđenu ocenu ili verovatnoću da će korisnik u biti zainteresovan za stavku i. Cilj sistema je da ova predviđena vrednost bude bliža stvarnom interesu korisnika. Na osnovu ovih vrednosti sistem rangira stavke i korisniku prikazuje one sa najvišom predviđenom ocenom.

Da bi sistem za preporuku mogao da generiše preporuku, prolazi kroz nekoliko koraka. Prvi korak je prikupljanje podataka. Sistem beleži sve interakcije korisnika sa platformom, bilo da se radi o eksplicitnim ili implicitnim signalima. Na osnovu tih podataka gradi se profil korisnika koji predstavlja digitalni otisak njegovih interesovanja i navika.

Drugi korak je pronalaženje kandidata. Platforma najčešće ima ogroman katalog sadržaja, milione pesama, hiljade proizvoda ili stotine hiljada filmova i nije moguće rangirati svaku stavku

za svakog korisnika u realnom vremenu. Zbog toga sistem najpre sužava skup mogućih preporuka na manji broj kandidata koji su potencijalno relevantni.

Treći korak je rangiranje. Kada sistem ima skup kandidata primenjuje složenije modele koji svakom kandidatu dodeljuju ocenu relevantnosti za konkretnog korisnika. Ovi modeli uzimaju u obzir mnogo više faktora: kontekst, vreme, uređaj sa kojeg korisnik pristupa, trenutno raspoloženje koje se može odrediti iz nedavnih interakcija. Korisniku se na kraju prikazuje lista preporuka poređana po toj oceni.

Ceo ovaj proces se odvija u delićima sekunde, a sistemi se kontinuirano ažuriraju kako bi pratili promene u korisničkim interesovanjima. Nečija interesovanja nisu statična. Korisnik koji je mesecima imao interesovanja za jednu stvar može odjednom početi da istražuje druge stvari i dobar sistem za preporuku treba da to primeti i prilagodi se.

Kada je sistem izgrađen postavlja se pitanje kako znamo da li je dobar? Performanse sistema za preporuku mere se kroz nekoliko metrika. Preciznost (*precision*) govori koliki procenat preporučenih stavki je korisniku zaista bio relevantan, dok odziv (*recall*) meri koliko od svih relevantnih stavki je sistem uspeo da pronađe.

Pored ovih, često se koristi i RMSE metrika koja meri koliko su predviđene ocene blizu stvarnih, kao i metrike poput NDCG koje uzimaju u obzir i redosled preporuka jer nije svejedno da li se najbolja preporuka nalazi na prvom ili desetom mestu liste.

3. Analiza projekta

U ovom projektu izvršena je implementacija i poređenje tri sistema za preporuku anime serija u programskom jeziku Python. Korišćene su biblioteke NumPy, Pandas, Matplotlib, Scikit-learn, Implicit i TensorFlow.

NumPy biblioteka se koristi za rad sa nizovima i matricama. Koristi se za konstrukciju matrice korisnik-anime i računanju skalarnih proizvoda između vektora latentnih faktora. Pandas biblioteka omogućava manipulaciju podacima i njihovu analizu, a korišćena je za učitavanje, filtriranje i transformaciju skupova podataka. Matplotlib i Seaborn su korišćeni za kreiranje vizuelizacija distribucija ocena, aktivnosti korisnika i poređenja rezultata modela. Biblioteka Implicit pruža efikasnu implementaciju ALS algoritma optimizovanu za implicitne povratne informacije, dok je TensorFlow korišćen za izgradnju i treniranje Two Tower i Deep Learning modela.

3.1. Analiza skupa podataka

U ovom projektu korišćen je javno dostupan skup podataka preuzet sa Kaggle platforme koji sadrži informacije o ocenama anime serija. Podaci su organizovani u dva dokumenta: anime.csv koji sadrži metapodatke o anime naslovima i rating.csv koji sadrži ocene korisnika

	anime_id	name	genre	type	episodes	rating	members
0	32281	Kimi no Na wa.	Drama, Romance, School, Supernatural	Movie	1	9.37	200630
1	5114	Fullmetal Alchemist: Brotherhood	Action, Adventure, Drama, Fantasy, Magic, Mili...	TV	64	9.26	793665
2	28977	Gintama°	Action, Comedy, Historical, Parody, Samurai, S...	TV	51	9.25	114262
3	9253	Steins;Gate	Sci-Fi, Thriller	TV	24	9.17	673572
4	9969	Gintama'	Action, Comedy, Historical, Parody, Samurai, S...	TV	51	9.16	151266

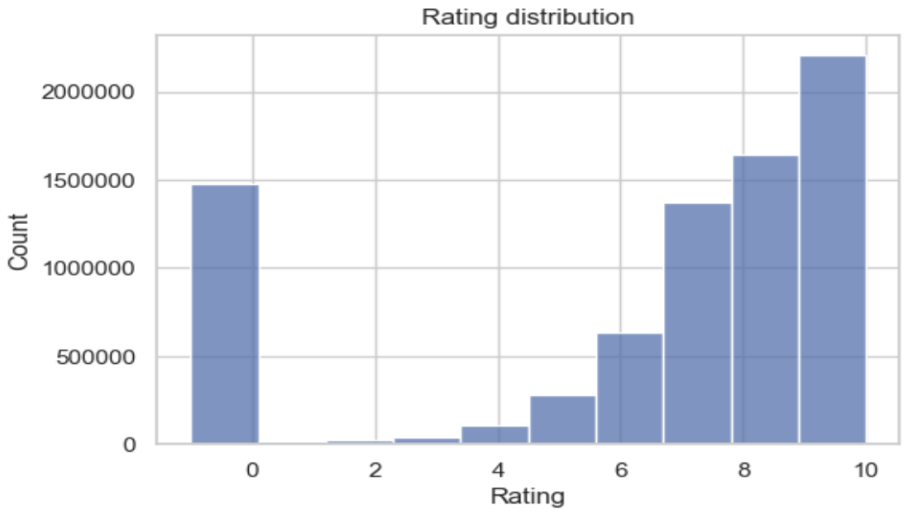
Slika 2. Isečak korišćenog skupa podataka anime.csv

Relevantni atributi skupa podataka su:

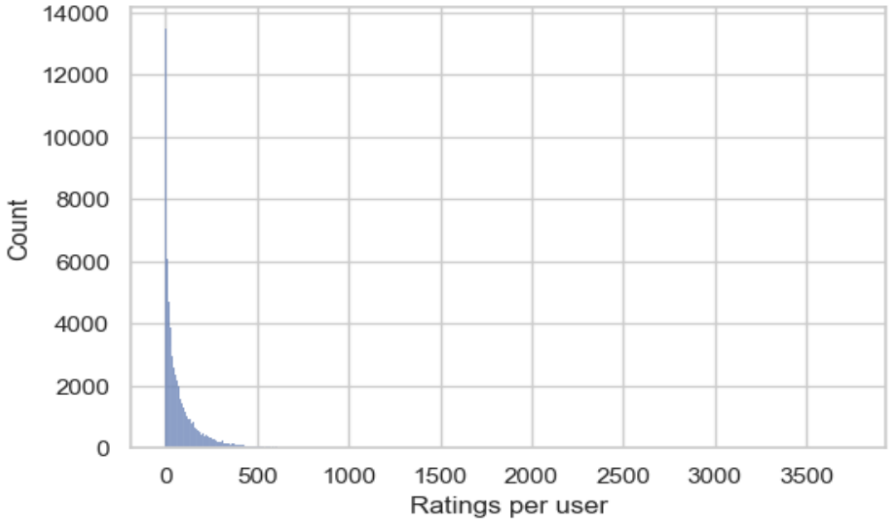
- **user_id** predstavlja jedinstveni identifikator korisnika koji je ostavio ocenu, jedan korisnik može oceniti veći broj anime serija
- **anime_id** predstavlja jedinstveni identifikator anime serije koja se ocenjuje
- **rating** predstavlja brojčanu ocenu koju korisnik dodeljuje seriji u opsegu od 1 do 10, vrednost -1 označava da je korisnik gledao seriju ali nije ostavio ocenu.

Nakon inicijalne analize utvrđeno je da skup podataka ne sadrži duplirane podatke ni nedostajuće vrednosti u ključnim atributima. Ocene sa vrednošću -1 su isključene iz analize kako bi se

treniralo isključivo sa eksplicitnim povratnim informacijama. Distribucija dodeljenih ocena pokazuje da su najučestalije ocene u gornjem delu skale. To sugeriše da korisnici češće ocenjuju serije koje su im se svidele. Dataset se sastoji od ocena korisnika u rasponu od 1 do 10, pri čemu su za potrebe modela kao pozitivne interakcije uzete samo ocene veće ili jednake 7. Korisnici sa manje od 5 pozitivnih interakcija i stavke sa manje od 20 pozitivnih ocena su isključeni iz analize, kako bi se osiguralo da model ima dovoljno podataka za učenje.

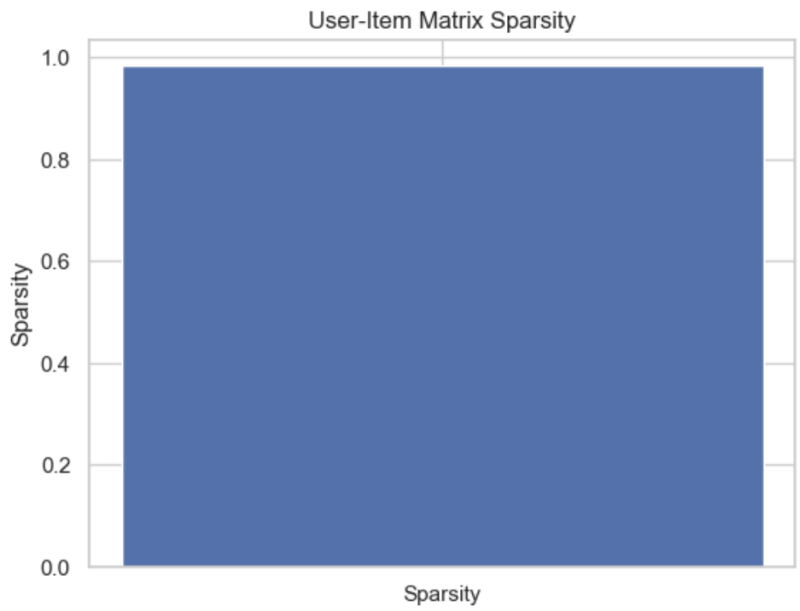


Slika 3. Disktribucija ocena



Slika 4. Distribucija ocena po korisniku

Pored distribucije ocena, analizirana je i gustina matrice korisnik-stavka. Nakon filtriranja, matrica interakcija ima sparsity vrednost koja ukazuje na to da je veliki deo mogućih kombinacija korisnik-anime ostao nepokriven. Ovo je tipična karakteristika realnih skupova podataka za preporuku i predstavlja jedan od ključnih izazova sa kojima se svi implementirani modeli moraju nositi.

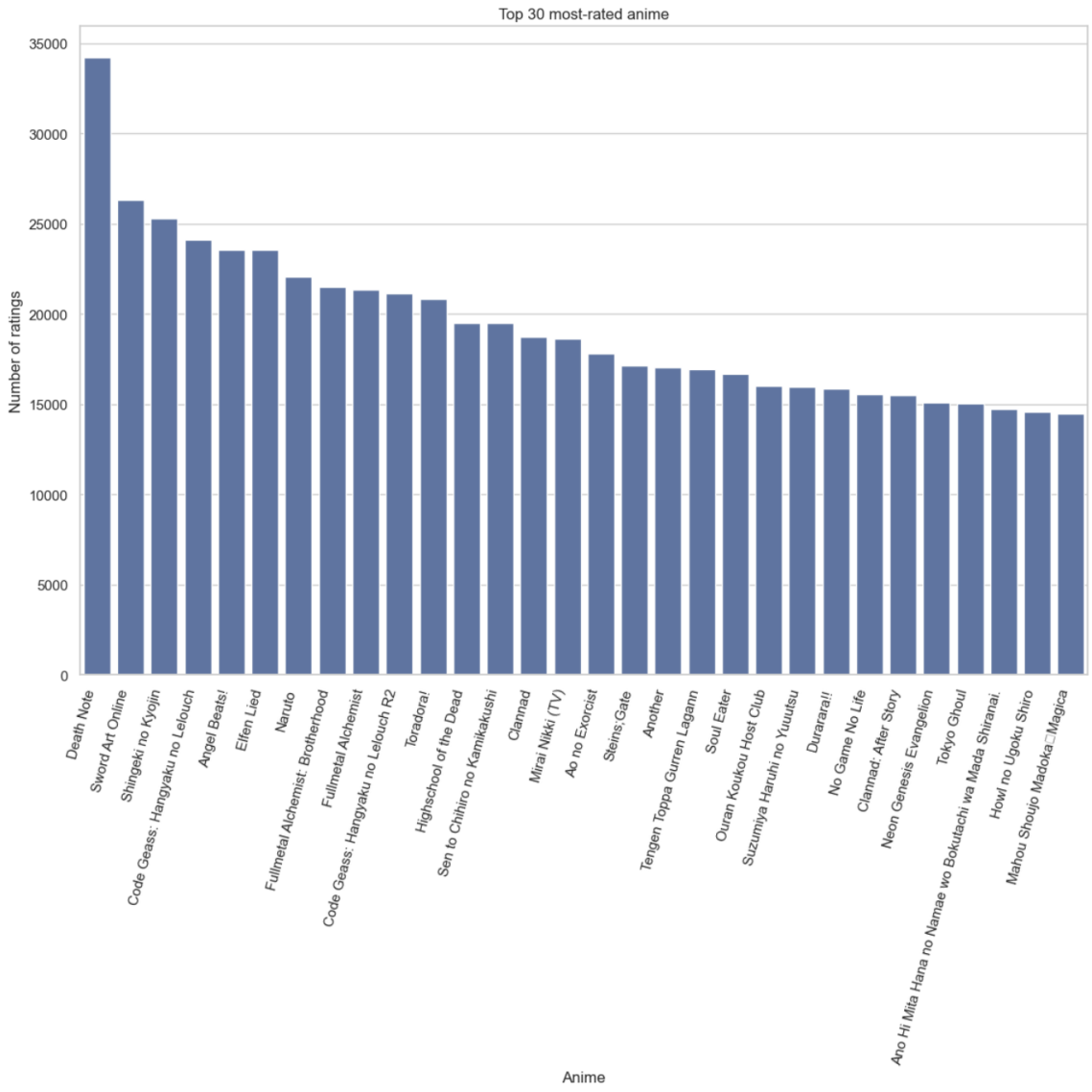



```
Sparsity: 0.9846740414446726
```

Slika 5. Sparsity vrednost korisnik-stavka matrice

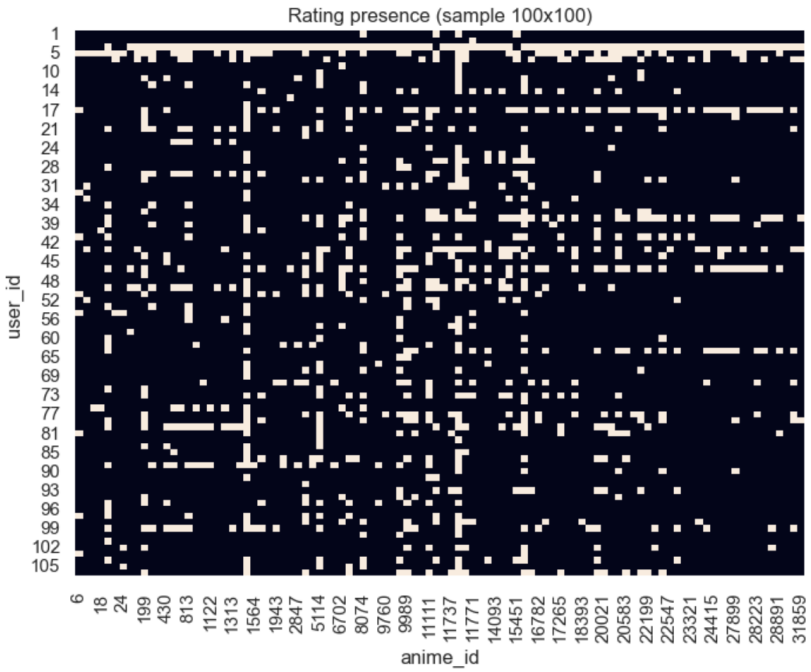
Analiza aktivnosti korisnika pokazuje izrazito neravnomernu distribuciju. Većina korisnika ima relativno mali broj ocena, dok mali broj veoma aktivnih korisnika dominira skupom podataka. Ovakva raspodela prati direktno utiče na kvalitet preporuka: modeli imaju mnogo više informacija o aktivnim korisnicima pa su im preporuke pouzdanije nego za korisnike sa malo interakcija.

Slično važi i za stavke. Analiza pokazuje da je nekoliko najpopularnijih anime serija ocenjeno od strane ogromnog broja korisnika dok velika većina naslova ima svega nekoliko desetina ocena. Takva raspodela pogoduje popularnim naslovima koji se češće pojavljuju u preporukama, što može dovesti do problema poznatog kao popularnosna pristrasnost tj. tendencije sistema da preporučuje ono što je već popularno, a ne nužno ono što je relevantno za konkretnog korisnika.



Slika 6. Top 30 anime serija sa najvećim brojem ocena

Vizuelizacijom prisustva ocena na uzorku od 100 korisnika i 100 anime serija potvrđena je retkost podataka. Heatmap prikaz jasno ilustruje da je velika većina polja u matrici korisnik-anime prazna. To znači da modeli moraju biti sposobni da generalizuju znanje iz malog broja poznatih interakcija na ceo katalog.



Slika 7. Heatmapa retkosti podataka

4. Matrix factorization

Matrix Factorization predstavlja jedan od najvažnijih i najuspešnijih pristupa u oblasti sistema za preporuku. Osnovna ideja je: interakcije između korisnika i stavki mogu se objasniti pomoću skrivenih, takozvanih latentnih faktora. Kod npr. preporuke anime, ti faktori mogu neformalno predstavljati sklonost ka određenim žanrovima, tempu ili radnji, ali ključna stvar je da oni nisu eksplicitno definisani i unapred poznati. Model ih sam otkriva iz podataka i to ga čini fleksibilnim i primenljivim u različitim domenima.

Polazi se od matrice ocena u kojoj redovi predstavljaju korisnike, a kolone stavke. Vrednosti u toj matrici su uglavnom nepoznate i većina korisnika je ocenila samo mali deo dostupnih stavki pa je cilj popuniti te praznine preciznije. To se postiže razlaganjem matrice na dve manje: latentne karakteristike korisnika i latentne karakteristike stavki.

Predviđena ocena dobija se kao skalarni proizvod odgovarajućih vektora, a model se trenira tako da te predikcije budu bliže stvarnim ocenama.

Prednost ovog pristupa je njegova efikasnost i relativna jednostavnost. Model zahteva manje računarskih resursa u poređenju sa dubokim neuronskim mrežama, lakše se trenira i interpretira, a istovremeno postiže solidne rezultate u mnogim praktičnim situacijama. Upravo zbog toga ostaje popularan i danas posebno u situacijama gde su resursi ograničeni ili kada je potrebno brzo postaviti funkcionalan sistem.

Cilj ALS algoritma je da pronađe dva skupa vektora: jedan za korisnike i jedan za stavke, čiji skalarni proizvodi što bolje aproksimiraju poznate interakcije. Broj dimenzija tih vektora je poznat kao broj latentnih faktora. Veći broj faktora omogućava modelu da uhvati suptilnije obrasce, ali povećava računsku zahtevnost i rizik od preopterećenja.

Ključna osobina ALS pristupa je način na koji rešava optimizacioni problem: naizmeničnim fiksiranjem vektora korisnika dok se optimizuju vektori stavki, i obrnuto. U svakom koraku problem se svodi na jednostavan sistem jednačina koji ima direktno rešenje. Ovaj postupak se ponavlja kroz iteracije sve dok rezultati ne postanu stabilni.

Međutim Matrix Factorization ima i jasna ograničenja. Pretpostavlja linearnu vezu između karakteristika korisnika i stavki. On nije u stanju da modeluje složenije obrasce ponašanja koji se javljaju u praksi. Pored toga teško integriše dodatne informacije kao npr. tekstualni opisi, slike ili vremenski podaci, a upravo te informacije mogu biti ključne za kvalitetnu preporuku. Ove slabosti su bile glavni motiv za razvoj naprednijih pristupa.

4.1. Praktična implementacija Matrix factorization modela

U praktičnoj implementaciji ovog rada Matrix Factorization je primenjen korišćenjem ALS (*Alternating Least Squares*) algoritma na skupu podataka koji sadrži ocene anime serija. Model je treniran sa 64 latentna faktora i faktorom pouzdanosti (*confidence*) alpha=40 i to je standardni pristup kada se radi sa implicitnim povratnim informacijama.

Umesto direktnih ocena model uči iz binarnih signala o tome da li je interakcija postojala ili nije. Treniranje je trajalo 15 iteracija, nakon čega su dobijene matrice latentnih faktora za korisnike i stavke dimenzija koje odgovaraju broju korisnika i anime naslova u filtriranom skupu podataka.

```python
from implicit.als import AlternatingLeastSquares

als_model = AlternatingLeastSquares(
    factors=64,
    regularization=0.01,
    iterations=15,
    random_state=42,
    num_threads=1,
)

alpha = 40.0

user_item_conf = (user_item_mat * alpha).tocsr()

als_model.fit(user_item_conf)

assert als_model.item_factors.shape[0] == n_items
assert als_model.user_factors.shape[0] == n_users

print("ALS training complete.")
```

```
100%|██████████| 15/15 [00:13<00:00,  1.11it/s]
ALS training complete.
```

Slika 8. ALS treniranje

Za evaluaciju je korišćena metrika Recall@10 i NDCG@10. Ovaj pristup evaluacije simulira realističan scenario u kome sistem treba da pronađe relevantnu stavku među većim skupom kandidata. ALS model je postigao Recall@10 od **0.8433** i NDCG@10 od **0.5918**. Visoka vrednost Recall@10 govori da je model u 84% slučajeva uspeo da pronađe relevantnu stavku među prvih 10 preporuka. Vrednost NDCG@10 ukazuje na to da model ne samo da pronalazi relevantne stavke već ih u većini slučajeva rangira na višim pozicijama liste. U praksi je to važno jer korisnici retko pregledaju preporuke ispod prvih nekoliko mesta.

4.2. Rezultati treniranja modela

Na slici 9. prikazan je primer top 10 preporuka koje je ALS model generisao za jednog korisnika. Model je preporučio serije poput Hunter x Hunter, One Punch Man i Tokyo Ghoul naslove koji spadaju u popularne anime. Preporučio je i Bakuman i Bakuman 3rd Season. Oni se pojavljuju kao odvojene stavke na listi. To je očekivano ponašanje sistema koji tretira svaku sezonu kao zaseban objekat. Ocene relevantnosti kreću se u rasponu od 1.057 do 1.197 što ukazuje na relativno ujednačenu distribuciju preferenca bez jednog dominantnog naslova.

```
Top-10 recommendations:
1. Hunter x Hunter (2011) (anime_id=11061, score=1.197)
2. One Punch Man (anime_id=30276, score=1.144)
3. Shaman King (anime_id=154, score=1.132)
4. Kiseijuu: Sei no Kakuritsu (anime_id=22535, score=1.109)
5. Tokyo Ghoul (anime_id=22319, score=1.097)
6. Zankyou no Terror (anime_id=23283, score=1.086)
7. Bakuman. (anime_id=7674, score=1.081)
8. Bakuman. 3rd Season (anime_id=12365, score=1.068)
9. Psycho-Pass (anime_id=13601, score=1.066)
10. Death Parade (anime_id=28223, score=1.057)
```

Slika 9. Top 10 preporuka Matrix factorization modela

5. Two Tower

Two Tower model predstavlja savremeniji pristup koji kombinuje ideju latentnih reprezentacija sa snagom neuronskih mreža. Njegova osnovna karakteristika je postojanje dve odvojene neuronske mreže (tornjeva). Jedna obrađuje podatke o korisniku, a druga podatke o stavci. Umesto da direktno modeluje interakciju između korisnika i stavke kao Matrix Factorization, ovaj pristup uči složenije reprezentacije kroz slojeve neuronske mreže.

Svaka mreža generiše vektorsku reprezentaciju poznatu kao embedding. Ovi vektori predstavljaju sažeti matematički opis korisnika i stavke u višedimenzionalnom prostoru. Što su dva vektora bliža jedan drugom u tom prostoru to je veća verovatnoća da će preporuka biti relevantna. Sličnost između vektora najčešće se meri kosinusnom sličnošću ili skalarnim proizvodom, a model se trenira tako da vektori korisnika i stavki koje mu se sviđaju budu bliži, dok su vektori irelevantnih stavki udaljeniji.

Ključna prednost ovog modela je fleksibilnost u pogledu ulaznih podataka. U korisnički toranj mogu se uključiti demografski podaci, istorija pretrage i ponašanje kroz vreme, dok toranj za stavke može koristiti tekstualne opise, kategorije, slike ili druge metapodatke. Ova mogućnost prirodnog uključivanja raznovrsnih izvora informacija čini Two Tower model posebno moćnim u situacijama gde samo podaci o interakcijama nisu dovoljni.

Two Tower modeli su posebno pogodni za velike sisteme jer omogućavaju unapred izračunavanje reprezentacija stavki. Na taj način se u realnom vremenu poredi samo korisnički vektor sa već postojećom bazom vektora stavki. Takav pristup značajno ubrzava proces preporuke i čini ga praktično primenljivim čak i na platformama sa milionima stavki.

U poređenju sa Matrix Factorization pristupom, on može modelovati složenije odnose između korisnika i stavki, ali zahteva više podataka za treniranje, veću računarsku snagu i pažljivije podešavanje hiperparametara. Pored toga, u osnovnoj varijanti model još uvek ne modeluje direktnu interakciju između karakteristika korisnika i stavki. To je problem koji rešavaju napredniji Deep Learning pristupi.

5.1. Praktična implementacija Two Tower modela

U praktičnoj implementaciji ovog rada, Two Tower model je izgrađen korišćenjem TensorFlow biblioteke. Oba tornja koriste embedding slojeve dimenzije 64, iste kao kod ALS modela, kako bi poređenje rezultata bilo fer. Korisnički toranj prima indeks korisnika i generiše njegov embedding vektor, dok toranj stavki na isti način obrađuje indeks anime serije. Model je treniran korišćenjem BPR (*Bayesian Personalized Ranking*) funkcije gubitka, koja direktno optimizuje rangiranje. Umesto da predviđa tačne ocene model uči da relevantne stavke rangira ispred

irelevantnih. Treniranje je trajalo 3 epohe sa 20 negativnih uzoraka po svakoj pozitivnoj interakciji.

Two Tower model je postigao Recall@10 od **0.5625** i NDCG@10 od **0.3043.** Postigao je slabiji rezultat u poređenju sa ALS modelom. Ovo na prvi pogled može delovati pogrešno, ali postoji nekoliko razloga za to. Model je treniran 3 epohe i to možda nije dovoljno. Pored toga, Two Tower model u ovoj implementaciji koristi samo identifikatore korisnika i stavki bez dodatnih metapodataka. Važno je napomenuti da ova razlika u rezultatima ne znači da je Two Tower arhitektura lošija od Matrix Factorization u opštem slučaju. U industrijskim primenama upravo ti dodatni podaci čine Two Tower modele moćnijim od jednostavnijih pristupa. Rezultati dobijeni u ovom radu pre svega odražavaju ograničenja konkretne konfiguracije, a ne inherentne slabosti same arhitekture.

```
history = bpr_model.fit(ds_bpr, epochs=3, verbose=1)
```

```
Epoch 1/3
19532/19532 ━━━━━━━━━━━━━━━━━━━━ 129s 7ms/step - loss: 0.3295
Epoch 2/3
19532/19532 ━━━━━━━━━━━━━━━━━━━━ 125s 6ms/step - loss: 0.3245
Epoch 3/3
19532/19532 ━━━━━━━━━━━━━━━━━━━━ 128s 7ms/step - loss: 0.3260
```

Slika 10. Two Tower treniranje

5.2. Rezultati treniranja modela

Na slici 11. prikazan je primer top 10 preporuka za istog korisnika kao u slučaju ALS modela. Two Tower je preporučio serije poput Steins;Gate, Bleach i Hellsing Ultimate koje su takođe popularni naslovi ali drugačijeg žanrovskog profila u poređenju sa ALS preporukama. One Punch Man i Psycho Pass se pojavljuju na listama oba modela pa se sugeriše da su ove serije veoma relevantne za ovog korisnika bez obzira na korišćeni pristup.

```
Top-10 recommendations:
1. Beelzebub (anime_id=9513)
2. Hyouka (anime_id=12189)
3. One Punch Man (anime_id=30276)
4. Tonari no Kaibutsu-kun (anime_id=14227)
5. Hellsing Ultimate (anime_id=777)
6. Guilty Crown (anime_id=10793)
7. Psycho-Pass (anime_id=13601)
8. Steins;Gate (anime_id=9253)
9. Bleach (anime_id=269)
10. Mirai Nikki (TV) (anime_id=10620)
```

Slika 11. Top 10 preporuka Two Tower modela

6. Deep learning

Deep Learning modeli predstavljaju najnapredniji pravac razvoja sistema za preporuku. Oni koriste višeslojne neuronske mreže sposobne da modeluju nelinearne i kompleksne obrasce u podacima koji su nedostupni jednostavnijim pristupima poput Matrix Factorization. Razlika leži u tome da duboke mreže ne pretpostavljaju nikakav unapred definisan oblik veze između korisnika i stavki. Tu vezu same otkrivaju iz podataka kroz proces treniranja.

Za razliku od jednostavnijih modela, duboke mreže mogu istovremeno obrađivati različite tipove podataka: tekst, slike, vremenske sekvence i strukturne informacije. To ih čini izuzetno pogodnim za moderne platforme gde su podaci heterogeni i gde sama istorija interakcija nije dovoljna za kvalitetnu preporuku.

Jedan od najpoznatijih pravaca razvoja jeste Neural Collaborative Filtering (NCF), koji kombinuje latentne reprezentacije sa višeslojnim perceptronima. Umesto da se vektori korisnika i stavka samo pomnože kao u Matrix Factorization, oni se prosleđuju kroz nekoliko gusto povezanih slojeva koji uče složenije oblike interakcije. Drugi važan pravac uključuje sekvencijalne modele koji analiziraju redosled korisničkih aktivnosti i pokušavaju da predvide sledeći korak. Ovakvi modeli su posebno korisni na platformama poput e-commerce sajtova gde redosled pregledanih proizvoda nosi važnu informaciju o nameri kupovine.

Prednost Deep Learning modela je visoka preciznost i sposobnost prilagođavanja kompleksnim zadacima, ali njihov nedostatak je velika računska zahtevnost, potreba za velikim količinama podataka i otežana interpretabilnost. Često je teško objasniti zašto je model preporučio određenu stavku pa to može biti problem u domenima gde je transparentnost važna.

```
history_dl = dl_model.fit(ds_dl, epochs=3, verbose=1)
```

```
Epoch 1/3
20508/20508 ━━━━━━━━━━━━━━━━━━━━ 150s 7ms/step - auc: 0.8885 - loss: 0.1340
Epoch 2/3
20508/20508 ━━━━━━━━━━━━━━━━━━━━ 158s 8ms/step - auc: 0.9171 - loss: 0.1199
Epoch 3/3
20508/20508 ━━━━━━━━━━━━━━━━━━━━ 140s 7ms/step - auc: 0.9310 - loss: 0.1111
```

Slika 12. Deep learning treniranje

6.1. Praktična implementacija Deep Learning modela

U praktičnoj implementaciji ovog rada, Deep Learning model je realizovan korišćenjem TensorFlow biblioteke. Model prima embedding vektore korisnika i stavke dimenzije 64, spaja

ih u jedan vektor konkatenacijom, a zatim ih prosleđuje kroz dva gusto povezana sloja od kojih je prvi sa 128 neurona i drugi sa 64 neurona. Nakon prvog sloja primenjen je Dropout od 0.2 kako bi se smanjio rizik od preopterećenja modela. Model je treniran korišćenjem binarne unakrsne entropije kao funkcije gubitka, sa 20 negativnih uzoraka po svakoj pozitivnoj interakciji, tokom 3 epohe.

Deep Learning model je postigao Recall@10 od **0.9170** i NDCG@10 od **0.6528** i to je najbolji rezultat od sva tri testirana pristupa. U poređenju sa ALS modelom Recall@10, poboljšanje je vidljivo, a razlika u NDCG@10 vrednostima sugeriše da MLP model ne samo da pronalazi relevantne stavke već ih i preciznije rangira na višim pozicijama liste. Ovi rezultati potvrđuju teorijsku pretpostavku da nelinearne arhitekture bolje hvataju složene obrasce u korisničkom ponašanju.

6.2. Rezultati treniranja modela

Na slici 13. prikazan je primer top 10 preporuka za istog korisnika. Model je preporučio naslove poput Tokyo Ghoul, Steins;Gate, Psycho Pass i Code Geass. Nekoliko naslova se ponavlja u poređenju sa ALS i Two Tower preporukama: Psycho Pass, Zankyou no Terror i Kiseijuu se pojavljuju u više lista.

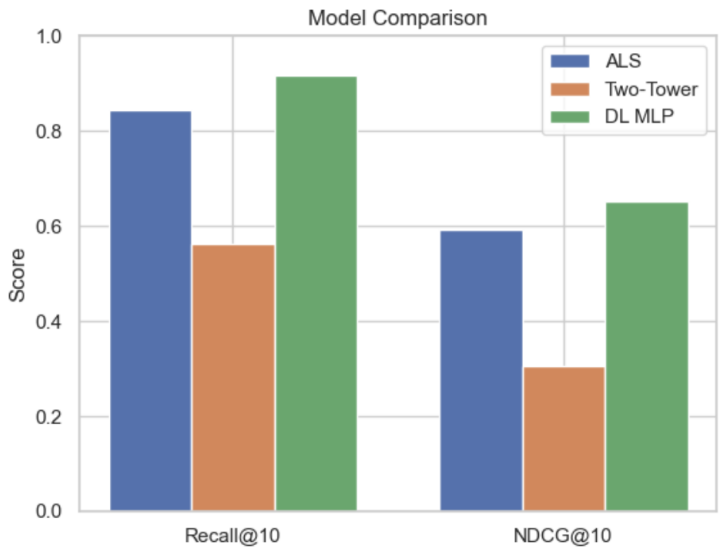
```
Top-10 recommendations:
1. Tokyo Ghoul (anime_id=22319)
2. Steins;Gate (anime_id=9253)
3. Mirai Nikki (TV) (anime_id=10620)
4. Psycho-Pass (anime_id=13601)
5. Angel Beats! (anime_id=6547)
6. Zankyou no Terror (anime_id=23283)
7. Kiseijuu: Sei no Kakuritsu (anime_id=22535)
8. Code Geass: Hangyaku no Lelouch (anime_id=1575)
9. Elfen Lied (anime_id=226)
10. Shigatsu wa Kimi no Uso (anime_id=23273)
```

Slika 13. Top 10 preporuka Deep learning modela

7. Poređenje

Deep Learning model postigao je najbolje rezultate za obe metrike. To potvrđuje da nelinearne arhitekture bolje primećuju složene obrasce u podacima. Matrix Factorization se pokazao kao dobra opcija uprkos svojoj jednostavnosti. Matrix Factorization je postigao rezultate bliske Deep Learning modelu i nadmašio Two Tower pristup. Two Tower model je u ovoj implementaciji pokazao najslabije rezultate. Uzrok toga može biti ograničen broj epoha treniranja ili odsustvo dodatnih metapodataka.

Rezultati pokazuju jasne razlike u performansama nakon implementacije i evaluacije sva tri pristupa. Na slici 14. prikazane su vrednosti Recall@10 i NDCG@10 za svaki model:



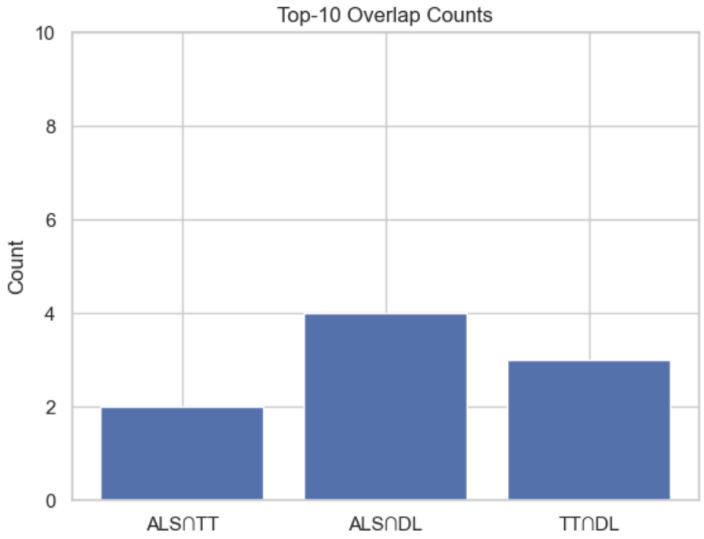
Slika 14. Poređenje Recall@10 i NDCG@10 sva tri modela

7.1. Poređenje preporuka anime serija

	ALS	Two-Tower	DL MLP
0	Hunter x Hunter (2011)	Beelzebub	Tokyo Ghoul
1	One Punch Man	Hyouka	Steins;Gate
2	Shaman King	One Punch Man	Mirai Nikki (TV)
3	Kiseijuu: Sei no Kakuritsu	Tonari no Kaibutsu-kun	Psycho-Pass
4	Tokyo Ghoul	Hellsing Ultimate	Angel Beats!
5	Zankyou no Terror	Guilty Crown	Zankyou no Terror
6	Bakuman.	Psycho-Pass	Kiseijuu: Sei no Kakuritsu
7	Bakuman. 3rd Season	Steins;Gate	Code Geass: Hangyaku no Lelouch
8	Psycho-Pass	Bleach	Elfen Lied
9	Death Parade	Mirai Nikki (TV)	Shigatsu wa Kimi no Uso

Slika 15. top 10 preporuke sva 3 modela

Posmatrajući preporuke za jednog konkretnog korisnika vidimo da sva tri modela preporučuju određene naslove. Anime serija Psycho Pass se pojavljuje na svim listama dok se Tokyo Ghoul, Zankyou no Terror i Kiseijuu pojavljuju na listama dva od tri modela. Ovo ukazuje na to da postoji grupa naslova koja je visoko relevantna za ovog korisnika bez obzira na korišćeni pristup dok se modeli razlikuju u preostalim preporukama.



Slika 16. Preklapanje top 10 preporuka

Posmatrajući žanrovski profil preporuka može se uočiti da ALS model favorizuje popularne akcione i avanturističke naslove što je očekivano s obzirom na to da se oslanja isključivo na obrasce interakcija i prirodno teži ka popularnim stavkama. Deep Learning model pokazuje nešto raznovrsniji izbor. Pored akcionih serija preporučuje i naslove iz žanrova drame i psihološkog thrillera što sugeriše da je uspeo da uhvati suptilnije obrasce u korisničkom ponašanju. Two Tower model u ovoj implementaciji daje preporuke sličnog žanrovskog profila kao Deep Learning, ali sa nižim stepenom preklapanja sa ostalim modelima, što ukazuje na drugačiji pristup rangiranju kandidata.

7.2. Brzina treniranja modela

Pored tačnosti, važan praktičan faktor pri odabiru modela je vreme treniranja i brzina generisanja preporuka. ALS algoritam se pokazao kao ubedljivo najbrži u fazi treniranja. 15 iteracija završeno je za svega nekoliko sekundi. Two Tower i Deep learning modeli zahtevaju znatno više vremena po epohi zbog propagacije gradijenata kroz slojeve mreže, što u produkcijskim sistemima koji se redovno pretreniravaju na novim podacima može biti odlučujući faktor.

Two Tower model se pokazao kao najsporiji u fazi treniranja iako koristi relativno jednostavnu arhitekturu. Generisanje negativnih uzoraka (20 po svakoj pozitivnoj interakciji) značajno povećava veličinu skupa za treniranje i produžava vreme po epohi. Deep Learning MLP model ima sličan pristup sa negativnim uzorcima, ali zahvaljujući efikasnijem batch procesiranju postiže nešto brže treniranje. ALS algoritam ostaje ubedljivo najbrži jer ne zahteva eksplicitno generisanje negativnih primera već negativan signal implicitno kodira kroz confidence matricu.

8. Zaključak

Sistemi za preporuku predstavljaju dinamičnu oblast istraživanja koja se kontinuirano razvija i čiji značaj u digitalnom svetu iz godine u godinu raste. Od jednostavnih faktorizacionih metoda do dubokih neuronskih mreža, napredak algoritama prati rast složenosti i količine podataka koje moderne platforme svakodnevno generišu.

Rezultati dobijeni u ovom radu potvrđuju teorijske pretpostavke o prednostima i ograničenjima svakog pristupa. Matrix Factorization se pokazao kao iznenađujuće robustan metod koji, uprkos svojoj jednostavnosti, postiže visoke performanse i ostaje relevantan zbog svoje stabilnosti, efikasnosti i lake implementacije. Two Tower model u osnovnoj varijanti nije uspeo da nadmaši Matrix Factorization, ali njegova arhitektura pruža odličnu osnovu za proširenje sa dodatnim metapodacima i složenijim slojevima. Deep Learning model postigao je najbolje rezultate potvrđujući da nelinearne arhitekture bolje hvataju složene obrasce u korisničkom ponašanju, ali po cenu veće računske zahtevnosti i potrebe za pažljivijim podešavanjem.

Izbor pravog pristupa u praksi nikada nije jednostavan. Zavisi od raspoloživih podataka, računskih resursa, zahteva za brzinom odgovora i koliko je važna interpretabilnost rezultata. U situacijama gde su resursi ograničeni, Matrix Factorization i dalje predstavlja solidan izbor. Kada su dostupni bogati metapodaci i dovoljno vremena za treniranje, Two Tower i Deep Learning modeli otvaraju mogućnosti koje jednostavniji pristupi ne mogu da dostignu.

9. Literatura

[1] Koren, Y., Bell, R., Volinsky, C. Matrix Factorization Techniques for Recommender Systems https://datajobs.com/data-science-repo/Recommender-Systems-%5BNetflix%5D.pdf

[2] Ricci, F., Rokach, L., Shapira, B. Recommender Systems Handbook https://edyaaleh.files.wordpress.com/2016/02/recommendersystemshandbook.pdf

[3] Goodfellow, I., Bengio, Y., Courville, A. Deep Learning https://www.deeplearningbook.org/

[4] TensorFlow dokumentacija https://www.tensorflow.org/docs

[5] PyTorch biblioteka https://docs.pytorch.org/docs/stable/index.html